

Digital Control II: A PID Controller in Discrete Time

From the textbook PID transfer function to a difference equation a microcontroller can run.

Ogata, *Modern Control Engineering*, digital PID.

A real PID loop runs on a computer: every sample it reads the error, updates its integral and derivative estimates, and writes a new command. In this tutorial you will:

- discretize a continuous PID controller with `c2d`,
- read off the resulting **difference equation**,
- close the loop in discrete time and compare with the continuous design, and
- see how the sample time changes the achievable performance.

Run with `publish('DiscretePID.m')`, or step through with **Ctrl+Enter**.

Contents

- Plant and a continuous PID design
- Discretize the controller (emulation design)
- The difference equation the computer actually runs
- Close the loop in discrete time
- Before vs. after: the sample time matters
- Summary
- Try it yourself

Plant and a continuous PID design

Plant $G(s) = \frac{1}{(s+1)(s+3)}$. We first design a continuous PID with a **filtered** derivative,

$$G_c(s) = K_p + \frac{K_i}{s} + \frac{K_d s}{T_f s + 1},$$

that gives a good continuous response. The derivative filter T_f keeps the controller proper: a pure $K_d s$ term has unbounded high-frequency gain, which a real controller cannot realize and which a discretization turns into a destabilizing ringing mode.

```
G = tf(1,conv([1 1],[1 3]));  
Kp = 8; Ki = 12; Kd = 2; Tf = 0.05;    % Tf: derivative-filter time constant  
Cc = pid(Kp,Ki,Kd,Tf);  
Tc = feedback(Cc*G,1);
```

Discretize the controller (emulation design)

The **emulation** workflow designs in continuous time, then discretizes the controller. Tustin is the usual choice for PID: it preserves the integrator and approximates the phase well.

```
T = 0.05;                % sample time (seconds)  
Cd = c2d(Cc, T, 'tustin');  
fprintf('Discrete PID controller C(z):\n'); Cd    %#ok<NOPTS>
```

Discrete PID controller C(z):

Cd =

$$K_p + K_i * \frac{T_s*(z+1)}{2*(z-1)} + K_d * \frac{1}{T_f + T_s/2*(z+1)/(z-1)}$$

```

with Kp = 8, Ki = 12, Kd = 2, Tf = 0.05, Ts = 0.05

Sample time: 0.05 seconds
Discrete-time PIDF controller in parallel form.

```

The difference equation the computer actually runs

The numerator/denominator of $C(z)$ are the coefficients of the recursion executed every sample:

$$u_k = b_1 e_k + b_2 e_{k-1} + b_3 e_{k-2} - a_2 u_{k-1} - a_3 u_{k-2}.$$

```

[bz,az] = tfdata(Cd,'v');
fprintf('u[k] = %.3f*e[k] %.3f*e[k-1] %.3f*e[k-2] %.3f*u[k-1] %.3f*u[k-2]\n', ...
        bz(1),bz(2),bz(3), -az(2), -az(3))

```

```

u[k] = 34.967*e[k] -63.800*e[k-1] +29.233*e[k-2] +1.333*u[k-1] -0.333*u[k-2]

```

Close the loop in discrete time

Discretize the plant too – with ZOH, since a physical hold sits between the computer and the plant – and close the loop.

```

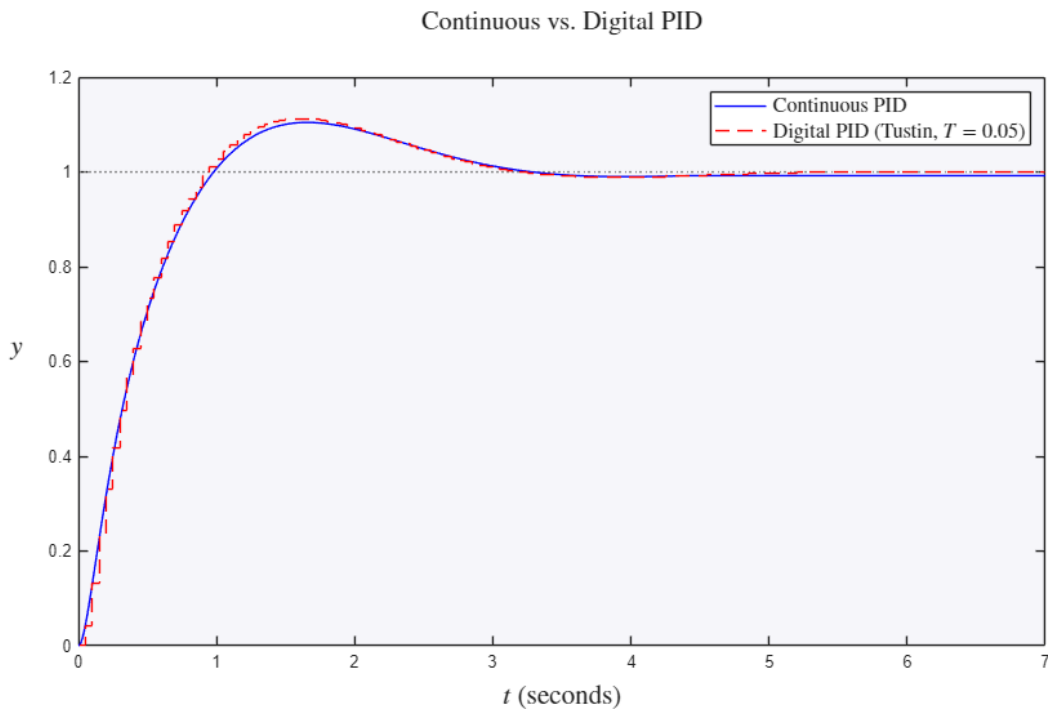
Gd = c2d(G, T, 'zoh');
Td = feedback(Cd*Gd, 1);

```

```

figure
step(Tc,'b', Td,'r--')
legend('Continuous PID','Digital PID (Tustin, $T=0.05$)','Interpreter','latex','FontSize',12)
title('Continuous vs. Digital PID','Interpreter','latex','FontSize',15)
ylabel('$y$','Interpreter','latex','FontSize',16); set(get(gca,'YLabel'),'Rotation',0)
xlabel('$t$','Interpreter','latex','FontSize',16)

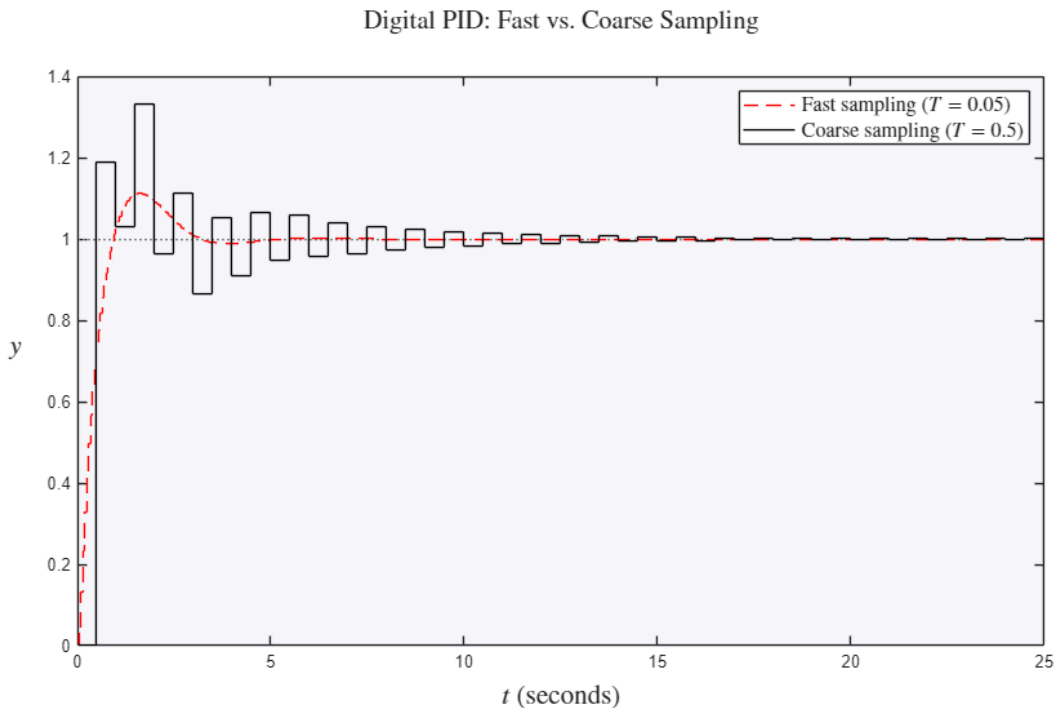
```



Before vs. after: the sample time matters

Emulation only works if you sample fast enough. Re-discretize at a much coarser rate and watch the extra phase lag erode the damping: the same controller now overshoots far more (still stable here), and pushed a little further it would go unstable.

```
T2 = 0.5;
Cd2 = c2d(Cc,T2,'tustin'); Gd2 = c2d(G,T2,'zoh');
Td2 = feedback(Cd2*Gd2,1);
figure
step(Td,'r--', Td2,'k')
legend('$T=0.05$', 'Coarse sampling ($T=0.5$)', 'Interpreter','latex','FontSize',12)
title('Digital PID: Fast vs. Coarse Sampling','Interpreter','latex','FontSize',15)
ylabel('$y$', 'Interpreter','latex','FontSize',16); set(get(gca,'YLabel'),'Rotation',0)
xlabel('$t$', 'Interpreter','latex','FontSize',16)
```



Summary

- Design continuous, then `c2d(C,T,'tustin')` to emulate the PID digitally.
- $C(z)$ is just a difference equation in past errors and commands.
- Sample fast: coarse sampling adds phase lag and can destabilize an otherwise healthy design.

Next: `DeadbeatControl.m` uses the discrete domain to settle in a finite number of steps – something continuous control cannot do.

Try it yourself

- Rebuild the controller with `'zoh'` instead of `'tustin'` and notice the integrator approximation shift slightly at the fast rate.
- Push `T2` past about 0.55 and watch the coarse-sampled loop lose stability – the step response starts to diverge.